



WX Claude Code

Conversão governada de projetos WINDEV, WEBDEV e WINDEV Mobile para outra plataforma. Questionário, gates com aprovação humana, equipe WLanguage sobre o Help, PMO com Scrum, Kanban e PDCA, e o contexto da primeira sessão do Claude Code gerado das respostas.

VERSÃO 3.32.0 · MEDIDO EM 2026-09-06

Em números

94

AGENTES

10 + 40

PAPÉIS COM PDCA

85

TESTES DE REGRESSÃO

31

SCRIPTS PYTHON

14

BLOCOS DO QUESTIONÁRIO

59

PROVAS EM SESSÃO REAL

29

CENAS NO VÍDEO

15.065

LINHAS DE PYTHON

Cada número é contado no repositório por `docs/dossie/numeros-do-plugin.py`; a tabela completa está em `docs/relatorio-do-plugin.md`.

Como um projeto atravessa o plugin

Questionário

bloco 0, A–L

Pré-flight G0

anexos conferidos

Inventário G1-G3

matriz, decisões

Piloto G4

golden master

Ondas G5-G6

papéis A–J, PDCA

Cutover G7

relatório, exportação

O questionário tem o bloco 0 da empresa (16 itens), as letras A a J, K do ambiente (9 itens) e L do contexto (6 itens); a letra F sozinha tem 14 subperguntas de qualidade de ERP. Dele saem até 102 arquivos: manifesto, configuração, respostas legíveis, empresa, processo, ambiente com instalador e SQL dos papéis, prompts de kickoff e prototipação, `INDEX_FILES.md`, o `.claude/` do projeto, `Dockerfile` e `compose`. O PMO fecha cada sprint com o relatório de onze seções e a exportação entrega o projeto organizado, com hashes, na pasta do usuário.

Regras que o projeto aprendeu medindo

Senha nunca em texto puro

o questionário guarda só o nome da variável; o gerador recusa chave senha/token com valor e valores com formato de token; a sessão real com a senha colada não a repetiu.

Valor de questionário é vetor de injeção

tudo que vira bash, SQL, YAML ou JSON passa por validação antes de gravar; identificador é identificador, porta é inteiro; teste com onze injeções.

Número visível sai de um gerador

os deste dossiê vêm de `numeros-do-plugin.py`; a página para investidores já ficou quatro versões com 34 agentes e 12 testes por ter sido digitada.

Interface só se prova exercitando

cada funcionalidade nova teve sessão real gravada; uma delas achou o `.env.exemplo` caindo no filtro de `.env`, outra o manifesto dizendo missing para dados que existiam.

O que não se mede fica INDISPONÍVEL

painel, relatório e medidor de ambiente nunca mostram zero no lugar do desconhecido.

Guarda nova entra pedida, não imposta

a licença trava só os scripts e a escrita em `.wx-migration`; o resto do Claude Code segue; o hook custa 54 ms medidos.

Skill que não aparece na listagem não existe.

As oito skills de ERP vieram com descrições de até 900 caracteres, e a impeccable já tinha provado que 895 some da listagem de uma sessão nova. Entraram com 150, os originais guardados ao lado; a prova é a sessão listar as onze pelo nome.

O que foi provado em sessão real

Cada linha é um print em `docs/prints/`: saída real de uma sessão do Claude Code ou de um script, sem edição. O vídeo de uso (3 min 38 s, 29 cenas) reproduz as mesmas capturas.

PRINT	ORIGEM
01	<code>`claude plugin validate`</code> no plugin e no marketplace, mais o validador offline em modo estrito
02	sessão <code>`-p`</code> pedindo a lista de skills e agentes com prefixo <code>`wx-claude-code:`</code>
03	três turnos reais (<code>`claude -p`</code> e <code>`-c`</code>): pergunta A, resposta, confirmação e pergunta B, resposta «não tenho», pergunta C

PRINT	ORIGEM
04	`aplicar_questionario.py` sobre respostas de exemplo e o `wx_preflight.py` real (BLOCKED por PDF de mentira, como devia)
05	o `DESIGN.md` que a letra F gera
06	`query_wlanguage_help.py --verify` e `--query HReadSeekFirst`
07	`claude -p "/wx-claude-code:laudo-tokens fase-1"` com fonte de sessões marcada INDISPONÍVEL
08	`pmo.py iniciar/gastar/status` e `rotear_modelo.py` com subida, rebaixamento a 85 %, bloqueio a 105 % e fallback
09	sessão `-p` com `Agent` liberado: dois símbolos delegados aos `wl-hfsql-specialist` e `wl-communication-specialist`, com o member do Help de cada um
10	`pmo.py sprint abrir`, dois ciclos `pdca` (um infrutífero, recusado sem `--proxima`), `kanban` com WIP e `sprint fechar` com resumo de doze seções
11	sessão `-p` com `Agent` liberado: `/wx-claude-code:pmo status` delegado ao agente do PMO, que rodou os scripts e apontou a dessincronia entre plano e sprint
12	o painel gerado sozinho por `pmo.py sprint fechar` num projeto com bloco 0, lacunas, decisões, riscos, PDCA e roteamento: o relatório de onze seções, o kanban e a base, aberto no Chromium com `prefers-color-scheme: light`
13	sessão `-p` e `-c`: letra H com «não sei a linguagem», sinais, três opções com a recomendada primeiro
14	o exemplo `estoque-wx`: `aplicar_questionario.py`, G0 `CONDITIONAL` sem erros, `extrair_pdf.py`, `golden.py` 9/10
15	`sprint abrir` com `--item ID:PAPEL`, `kanban` com `[C dba]` etc., `sprint fechar` e `entregar` listando o zip; `tecnicas-aplicadas.md` de dentro do zip
16	`aplicar_questionario.py` no exemplo com as treze subperguntas de F; trechos do `DESIGN.md` (grids, tabela de botões, posição, fundo) e do `PRODUCT.md`
17	três turnos reais: abertura do wizard com o item 0.1 sozinho, resposta, confirmação e 0.2, resposta e 0.3
18	sessão limpa em que o usuário cola a senha do GitHub no meio da resposta: o agente não a reproduz de forma nenhuma, pede para revogar e registra só URL e usuário (a senha digitada foi mascarada no print; a saída do agente é a real, e o `grep` pelo valor devolve 0)

PRINT	ORIGEM
19	sessão <code>`-p`</code> e <code>`-c`</code> com <code>`--allowedTools Read`</code> : letra H com os quatro sinais, três opções com a recomendada primeiro, e o processo de conversão para Python lido de <code>`references/perfis-de-destino.md`</code> , peça por peça
20	sessão <code>`-p`</code> e <code>`-c`</code> com leitura liberada: letra F pede a tela principal como modelo (F0); o agente abriu as capturas de <code>`inputs/screenshots/`</code> , propôs o que preservar a partir do que viu, registrou F0 e só então passou à F1
21	<code>`licenca.py verificar`</code> sem serial, sessão <code>`-p`</code> do PMO recusando pelo contexto do hook <code>`SessionStart`</code> , <code>`licenca.py instalar`</code> com um serial de demonstração e a mesma sessão rodando o PMO em seguida
22	sessão nova, sem contexto, com leitura liberada: perguntada sobre o aprovador e o prazo, achou os dois em <code>`respostas_questionario.md`</code> pelo <code>`CLAUDE.md`</code> , sem perguntar de volta
23	sessão limpa em que o usuário adianta a letra K2 com a senha do root do PostgreSQL: o agente não a reproduz de forma nenhuma, registra banco, superusuário e papéis, e diz que vai pedir só o nome da variável (a senha digitada foi mascarada no print; o <code>`grep`</code> pelo valor na saída do agente devolve 0)
24	dois turnos reais (<code>`-p`</code> e <code>`-c`</code>): K7 «sim» ao n8n, o item K7.1 sozinho, resposta, e o K7.2 avisando que sem K2 o banco do n8n pode ser SQLite ou PostgreSQL
25	sessão nova com leitura liberada, num projeto recém-aplicado: perguntada o que ler e qual o escopo da v1, listou a ordem de leitura, os cinco requisitos do kickoff, a estratégia, e recusou escrever código por falta de G0; de quebra achou um conflito real entre os anexos e o manifesto
26	sessão <code>`-p`</code> com <code>`Agent`</code> e <code>`Bash`</code> : <code>`/wx-claude-code:pmo exportar`</code> grava o projeto organizado na pasta pedida e explica o que ficou de fora; achou um defeito real (o <code>`env.exemplo`</code> caía no filtro de <code>`env`</code>), corrigido com teste
27	sessão <code>`-p`</code> num projeto com log antigo: o hook <code>`SessionStart`</code> rodou o zelador e a sessão mostra o registro com bytes medidos
28	dois turnos reais (<code>`-p`</code> e <code>`-c`</code>): a resposta abre com a identificação <code>`Bloco0001-SP00002-Análise da base de dados`</code> · <code>`data`</code> injetada pelo hook; no segundo turno o agente se recusa a inventar o objetivo da sprint nova e antecipa a identificação <code>`SP00003`</code>
29	sessão real: perguntada qual agente entra num PDCA infrutífero, apontou o Pesquisador (equipe B), achou o pedido pendente em <code>`pmo/pesquisas.md`</code> com a hipótese morta e a próxima, e deu o status por agente recusando inventar estado para quem não registrou
30	duas sessões reais: a pergunta sobre a estratégia foi respondida citando <code>`processo-de-conversao.md#L5-L9`</code> , o localizador que o RAG injeta; pedido para apagar um anexo, o agente recusou pela regra antes de o hook precisar negar (o hook está provado por teste, com <code>`stdin`</code>)

PRINT ORIGEM

- 31 sessão real num projeto recém-aplicado: perguntada o que faz ``HReadSeekFirst``, consultou o Help pelo tema que o hook do RAG apontou, citou a página com id e hash, e separou semântica técnica de regra de negócio, como o ``CLAUDE.md`` gerado manda
- 32 ``claude --plugin-dir wx-claude-code -p "liste as skills"`` (skills globais do ambiente omitidas do print): as onze skills do plugin aparecem, as oito de ERP com descrição de até 150 caracteres; perguntada por partidas dobradas e estorno, apontou ``erp-accounting``
- 33 sessão real num projeto com L6 = sim: leu ``CLAUDE.md`` e ``AGENTS.md``, listou módulo → skill, resumiu a ADR 0002 (sem multiempresa na v1) e carregou a ``erp-inventory``, citando INV-01 e INV-06 sobre saldo
- 34 sessão real na 3.18.0: leu ``docs/skills-recomendadas.md`` e separou as que cabem (Rust, PostgreSQL, React) das que ficaram fora (Supabase não instalado, sem multiempresa); citou ``NUMERIC(19,4)`` e ``TIMESTAMPTZ`` do modelo de dados; explicou que cada mitigação STRIDE vira ``SEC-*`` e teste
- 35 sessão real na 3.19.0: leu ``artefatos/CATALOGO.md`` e o ``CLAUDE.md``, distinguiu o artefato arquivado dos três só declarados e recusou usar o que não foi submetido; explicou o hash como prova; carregou a skill de PHP e citou ponto flutuante em dinheiro e comparação frouxa, com ``BigDecimal`/centavos` e ``DECIMAL(19,4)``
- 36 sessão real na 3.20.0: listou os dezessete comandos, rodou o ``listar_perguntas.py`` (59 perguntas, aprovador 0.16, tela modelo F0), e respondeu que legado só PHP com destino Elixir é aceito — dizendo como preencher e que o caso pétreo continua sendo WLanguage para outra linguagem
- 37 sessão real na 3.21.0: leu o plano do K8 e citou RPO 15 min, RTO 120 min e a data da última restauração testada com ``arquivo:linha``; respondeu que a réplica assíncrona não substitui o backup, citando o próprio documento; e achou sozinha as 60 respostas por id, dizendo que nenhuma estava pendente
- 38 sessão real na 3.22.0: converteu o PDF do código em markdown pelo script do plugin e citou a página 1 para a regra do desconto; depois leu o registro de operações e explicou as duas únicas entradas com erro — negativas do hook, não falha de script. Foi esse registro que revelou o excesso de bloqueio corrigido no mesmo dia
- 39 a bateria rodada pelo próprio Claude Code numa sessão real: 49 testes OK, validador estrito com 13 skills, 94 agentes, zero erros e zero avisos, e ``claude plugin validate`` passando; os tempos são os medidos na sessão
- 40 sessão real na 3.23.0: rodou ``instalar.sh --conferir`` e relatou os cinco passos com o que achou em cada um, confirmando que nada foi instalado; e leu o ``FONTES.md`` dizendo os 212 arquivos, as 26 mil linhas e o que vem no pacote sem ser fonte. Foi essa sessão que reparou no ``/tmp/wx-validacao.json`` deixado para trás, corrigido no mesmo dia

PRINT ORIGEM

- 41 sessão real na 3.24.0: carregou a `ui-ux-pro-max` recém-vendorizada, citou a origem e a licença MIT pelo `NOTICE.md`, e trouxe dois dados da base local com o arquivo e a linha (`styles.csv`, `ux-guidelines.csv`); depois leu o `PRE-REQUISITOS.md` e explicou o que acontece sem `pypdf`
- 42 sessão real na 3.25.0: testou o instalador com o manifesto ausente e com o CLI fora do PATH, mostrou o que ele oferece em cada caso, e conferiu depois — com `ls` e `command -v` — que nada foi instalado sem aprovação; citou a linha da função que recusa sozinha quando não há terminal
- 43 sessão real na 3.27.0: carregou a skill `modelos-locais`, citou a origem e a licença do Magnitude e que o plugin não o redistribui; rodou o roteador nos três cenários (serviço no ar, fora do ar, classe de decisão) e conferiu que o tratamento de `dado-pessoal` na skill bate com o código, citando as linhas
- 44 sessão real na 3.28.0: rodou o `tests/fluxo.py`, relatou os treze passos com o tempo de cada um e explicou o que ele prova que a bateria não prova — a ligação entre as peças, citando os três defeitos históricos de «peça certa, ligação errada» e a distinção entre código esperado por contrato e falha de verdade
- 45 `instalar.sh --conferir` na 3.29.0 com os cinco passos e o `claude plugin validate` logo depois: em modo conferir nada é instalado, e o passo 3 traz 21 skills e 94 agentes medidos pelo validador
- 46 a bateria pesada `tests/cenarios.py` rodada de verdade: doze cenários, 12/12, com o tempo medido de cada um — os caminhos que um cliente real traz (sem licença, PDF que é foto, legado que nunca foi WX, resposta que se contradiz)
- 47 a procedure WLanguage `CalculaDesconto` como ela sai do PDF do legado pelo `pdf\_para\_markdown.py`, com a página 1 e o sha256 do PDF preservados
- 48 o Rust gerado por uma sessão real a partir da página 1: cada regra de negócio no código traz a origem citada, e o teto de 15%/25% não foi inventado
- 49 a mesma sessão rodando `cargo test` (6 testes) e explicando o que mudou de semântica na tradução — o `RETURN -1` que virou `Result`, o dinheiro em centavos e o `>` mantido literal
- 50 `licenca.py verificar` e o hook `PreToolUse` num ambiente **sem serial**: o verificador sai com código 3 e o hook nega o próprio script do plugin, com a razão
- 51 a liberação inteira: a impressão da máquina, o serial assinado com a chave privada (do lado de quem vende), a instalação no cliente, o verificador dizendo válida e o mesmo comando de antes passando
- 52 a regra BR-101 como ela está no legado PHP de 2009 — `calcula\_encargos`, com `strtotime`, `round` e as constantes de multa e juros

PRINT	ORIGEM
53	o questionário aplicado num projeto PHP puro e o G0 sobre ele: <code>`CONDITIONAL`</code> , zero erros, e o código-fonte listado como evidência central com as linhas medidas de cada arquivo
54	o Rust gerado por uma sessão real a partir do PHP, com <code>`regras.php:19-33`</code> citado no cabeçalho do módulo e o golden master nomeado como prova
55	<code>`cargo test`</code> com os três casos do golden master, e a sessão dizendo o que mudou de semântica (<code>`strtotime`</code> → dias de calendário) e o que ela não converteu por não ser dela decidir (o <code>`f64`</code> no lugar de decimal)
56	<code>`php capturar-golden.php`</code> : o esperado do exemplo PHP sai rodando o próprio legado, 14 casos, com a versão do PHP e a data registradas no arquivo
57	sessão nova sem contexto no projeto PHP: achou o aprovador e o prazo nas respostas por id, a baixa de título sem transação em <code>`titulo_baixar.php:15-21`</code> , e a view <code>`v_inadimplencia`</code> que nenhum PHP consulta
58	<code>`exportar_projeto.py`</code> com as sete pastas numeradas e o SHA-256 de cada um dos 38 arquivos, e o <code>`registro.py resumo`</code> com as 30 operações que o plugin fez naquele projeto
59	a bateria pesada com treze cenários, o de número 13 sendo este legado PHP inteiro atravessando o G0

O que ainda não foi provado

- Nenhum projeto WINDEV real passou pelos gates G1 a G7 de ponta a ponta; o exemplo ESTOQUE é sintético.
- Os scripts de ambiente são bash; não há PowerShell, e o público usa Windows.
- A licença é dissuasão por hook; servir corpus e agentes de um servidor ficou para depois, por decisão do dono.
- O custo em tokens do questionário inteiro numa sessão real não foi medido.
- O questionário não pausa nem retoma; com mais de setenta itens, isso pesa.

Versões

VERSÃO	DATA	O QUE ENTROU
3.31.1	2026-09-05	--inverter, porque grep sai 1 quando não acha
3.31.0	2026-09-05	seis portões de governança, todos entrando pedidos
3.30.0	2026-09-05	exemplo de legado PHP, e o G0 que ainda supunha WINDEV
3.29.0	2026-09-05	bateria pesada de cenários, com relatório gerado rodando a bateria
3.28.0	2026-09-05	teste de fluxo, e o fluxograma mostrando as duas provas
3.27.0	2026-09-05	modelo local pelo Magnitude, como degrau abaixo do mais barato
3.26.0	2026-09-04	folha de referência dos comandos e das perguntas, gerada dos arquivos
3.25.1	2026-09-04	revisão geral — quatro páginas envelheciam caladas, e agora um comando as atualiza
3.25.0	2026-09-04	o instalador resolve o pré-requisito que falta, sempre pedindo aprovação
3.24.0	2026-09-04	UI/UX Pro Max vendorizada (MIT) e a lista de pré-requisitos medida
3.23.0	2026-09-04	instalador completo nas duas plataformas e o inventário das fontes
3.22.1	2026-09-04	o comando da licença mandava rodar subcomandos que nunca existiram
3.22.0	2026-09-04	PDF vira markdown citável, e toda operação do plugin deixa registro
3.21.0	2026-09-04	K8 (backup e replicação) fecha 60 perguntas, e as respostas viram documento dos agentes

VERSÃO	DATA	O QUE ENTROU
3.20.0	2026-09-04	legado E/OU, destino em qualquer linguagem, e um comando para cada recurso
3.19.0	2026-09-04	PHP nos dois sentidos e o bloco M dos artefatos submetidos
3.18.0	2026-09-04	o que a pesquisa no skills.sh acrescentou ao esqueleto, e por que as 27 skills não entram no plugin
3.17.0	2026-09-04	vídeo com a cena 24 (esqueleto de ERP) e dossiê com os números medidos
3.17.0	2026-09-04	oito skills de ERP e o esqueleto de projeto que o questionário gera (L6)
3.16.1	2026-09-04	o corpus WLanguage entra no CLAUDE.md gerado e no RAG por símbolo
3.16.0	2026-09-04	hooks que fazem valer as regras e um RAG local do projeto
3.15.0	2026-09-04	equipe prioritária de dez agentes, cada um com gatilho próprio
3.14.0	2026-09-03	blocos e sprints numerados, identificação obrigatória em cada interação, sprint zipada
3.13.0	2026-09-03	exportar o projeto organizado, zelador diário, relatório e dossiê medidos, vídeo com todas as provas
3.12.1	2026-09-03	revisão inteira — injeção no gerador, filtro de segredos, portão G0 e 50 descrições cortadas
3.12.0	2026-09-03	letra L — o contexto da primeira sessão sai pronto do questionário
3.11.0	2026-09-03	n8n integrado ao projeto (K7) e privilégios do instalador (K0)
3.10.0	2026-09-03	letra K — Rust/Cargo, PostgreSQL, MySQL, MariaDB, Supabase e GitHub no questionário
3.9.0	2026-09-03	o wizard pergunta o aprovador e grava todas as respostas em respostas_questionario.md
3.8.1	2026-09-03	relatório de onze seções gerado sozinho ao fechar sprint e na entrega

VERSÃO	DATA	O QUE ENTROU
3.8.0	2026-09-03	serial de ativação assinado com RSA, hooks de licença e marca d'água
3.7.2	2026-09-03	a tela principal do projeto entra como modelo visual (F0)
3.7.1	2026-09-03	H e I mostram o processo de conversão e perguntam a estratégia
3.7.0	2026-09-03	bloco 0 do questionário — empresa, projeto, prazo, orçamento e GitHub sem senha
3.6.1	2026-09-03	botões perguntados um a um — vocabulário, posição, ícone, cor e fundo
3.6.0	2026-09-03	a letra F vira oito subperguntas de ERP, cada uma ligada a um comando do Impeccable
3.5.0	2026-09-03	dez papéis com PDCA, backlog com dono e entrega zipada ao stakeholder
3.4.0	2026-09-03	para qual linguagem converter, WL_C# como perfil, prints e vídeo com o exemplo
3.3.0	2026-09-03	projeto de exemplo real, provas em código, portão G0 e testes de regressão
3.2.1	2026-09-03	pacote com o questionário letra a letra

Lido do `git log`: só os commits cuja mensagem começa pela versão.

Onde está cada coisa

ARQUIVO	O QUE É
MANUAL.md, docs/manual-de-uso.pdf	manual de oito capítulos
docs/relatorio-do-plugin.md	relatório medido do estado atual
docs/investidor/	página e PDF para investidores

ARQUIVO	O QUE É
docs/prints/, docs/video/	provas reais e o vídeo de uso
docs/relatorio-de-cenarios.pdf	relatório da bateria pesada, gerado rodando os doze cenários
docs/analise-aula-vibe-coding.md	o que a aula de vibe coding ensinou e o que não se copiou
docs/telas-licenca/	sete telas do fluxo de liberação de licença
licenca/LEIA-ME.md	o que o serial protege e o que não
skills/LEIA-ME-erp.md	as oito skills de ERP do pacote skills.sh e o esqueleto que L6 gera
exemplos/estoque-wx/	projeto de exemplo e teste de regressão do fluxo inteiro
tests/testes.py	85 testes; o validador estrito os roda

Gerado por docs/dossie/gerar-dossie.py em 2026-09-06; regenerar em vez de editar. Built to store. Engineered to scale.